Emotion Driven Human-AI Interaction

Cecilie Städe Kristensen
Aalborg University
Aalborg, Denmark
Cskr20@student.aau.dk

REFLECTION ON TECHNICAL AND DESIGNERLY CONSIDERATIONS

The goal with this prototype was to explore how human-AI interaction can be made emotionally supportive, by combining natural language, real-time feedback, visual cues, and a calm and minimal interaction space. This resulted in a Unity based conversational environment, connected to a Python backend which performs an emotion analysis and generates empathetic responses.

Unity handles all user interaction. The interface includes a chat scroll area, a user input field, and a 3D emotional avatar cube (00:05-00:14). The UI uses stronger colors and a minimal layout to create a calmer space. In UIController.cs, the input text is reset, and the placeholder is set to English at runtime, regardless of the system language (Figure 1). My computer is set to Danish, and this prevents Unity from automatically switching the placeholder to Danish.

```
46    // Start with an empty input string
47    if (input != null)
48    {
49        input.text = "";
50
51        // Force English placeholder text
52        var placeholder = input.placeholder as TMP_Text;
53        if (placeholder != null)
54        {
55            placeholder.text = "Type your message here...";
```

Figure 1 Placeholder and reset input field

This setup follows the general architecture from the course, where Unity manages interaction, and visualisation and Python handles all text understanding and AI processing.

First, the system uses VADER, a sentiment analysis toolkit. VADER returns a compound sentiment score between -1 and +1, which my system in emotion.py translates into emotional labels like; *happy*, *sad* and *neutral*.

Because VADER struggles to detect emotions like *stressed* and *angry*, I added a small sort of hybrid classifier based on hardcoded keyword rules (Figure 2). If the text contains words like *stress*, *stressed*, or *overwhelmed*, the classifier directly returns the emotion *stressed*. Each of the emotions is mapped to a specific RGB colour, which is later used on the avatar cube in Unity.

```
55      t = text.lower()
56      scores = analyzer.polarity_scores(text)
57      comp = scores["compound"]
58
59      # keywords for "stressed"
60      if any(w in t for w in ["stress", "stressed", "overwhelmed"]):
61          return "stressed", abs(comp)
62
63      # keywords for "angry"
64      if any(w in t for w in ["angry", "mad", "annoyed", "irritated"]):
65          return "angry", abs(comp)
66
67      # sentiment-based mapping VADER
68      if comp >= 0.4:
69          return "happy", comp
70      if comp <= -0.2:
71          return "sad", abs(comp)
72
73      return "neutral", abs(comp)
```

Figure 2 VADER and hardcoded emotions

The Python backend is a small custom TCP server in server.py that exchanges JSON messages with Unity in real time (00:19-00:25). Unity sends the user input to Python, where the system uses VADER and the hardcoded rules. After that the backend sends the user message, the detected emotion label, and a short history of the last messages for

context to a local LLM via Ollama. The LLM uses this to generate a short and empathetic English reply following a prompt. Python then sends a JSON response back to Unity including the emotion label, the RGB colour for the cube, and the LLM output, which is then displayed in the chat (00:25-00:50). The avatar cube is sort of a companion, that reflects the emotional states through colour and gentle movements. For example, *happy* makes a slight bounce, *sad* makes the cube sink, and *stressed* makes it wobble (00:47-00:58). These animations were chosen because they are expressive but with gentle movements, so they won't distract the user too much (see Figure 3 below for an example).

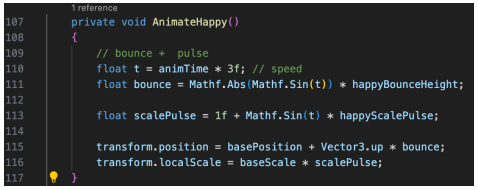

Figure 3 Cube movement (Happy)

The chat messages have a left/right separation, where AI messages appear on the left and user messages appear on the right (00:24-02:36). I chose to use a familiar messaging design, so that the user would feel more comfortable. This choice also keeps the chat scroll uncluttered and simple.

BEYOND COURSE BASICS

This project is built on the lecture examples, where Unity sends text to Python via a socket connection and then receives an AI generated output. However, my project goes a bit beyond the examples.

First, I choose to implement VADER, a sentiment rule-based analysis tool, which analyses the emotional tone of the text and gives positive/negative/neutral scores. I extended it by mapping the sentiment scores with emotional labels and combined it with hardcoded rules to target emotions like *stressed* and *angry* as well.

Other than that, I extended the integration of a local LLM via Ollama with my own structured system prompt and added conversation history handling to get a context aware response, and an emotional response. As well as a fallback if the model should stop working as a backup (USE_LLM = True), so it would use hardcoded replies. This was used at the beginning to test out my setup, before implementing the LLM.

And lastly, I implemented a 3D emotional avatar cube that visualizes emotion through different animation and colour transitions. The course introduced different animations. In addition to recognising the users emotional state from the input text, the prototype tries to stimulate the users affective state through the cube, by incorporating the design principles from the *Affective Computing lecture*.

CRITICAL OUTLOOK AND FUTURE EXTENSIONS

Given more time for development, the prototype could be extended both technically and designerly.

A technical extension could be done by replacing the avatar cube with a more expressive avatar, that could be able to make facial expressions based on the emotional feedback. This should be done in a subtle way, so it does not come across as mocking.

Another extension could be the implementation of speech-based interaction using Whisper from lecture 6, so that the user could speak naturally instead of typing. Text input should still be available for users with speech difficulties, or for anyone who simply just do not prefer to say it out loud.

From a designerly perspective, the colours and animations on the cube are hardcoded, and not user validated. According to lecture 9 *Affective Computing* the emotional feedback should ideally be personalised. So, a future extension could be allowing the user to edit and adjust the avatars behaviour and colours, and implement some type of machine learning, so it learns a specific style and pattern over time.

Lastly, on the AI side, the system could integrate a larger and more capable LLM, to get more of a long-term history. As well as get more advanced and improved quality of responses back.